

Recursion and Iteration

LEGAL NAME	<i>LEONARD-ADRIAN BUNEA</i>
PREFERRED NAME	ZOE MCFIFE
TIME SPENT	04:19:21

Table of Contents

Table of Contents	1
Table of Figures.....	2
Exercise 1 – Fibonacci.....	3
Solution Idea.....	3
Code.....	3
Test	5
Exercise 2 – Tower of Hanoi.....	7
Solution Idea.....	7
Code.....	7
Test	8
Exercise 3 – Decimal to Binary	10
Solution Idea.....	10
Code.....	10
Test	10
Exercise 4 – Number to Words	12
Solution Idea.....	12
Code.....	12
Test	15
Exercise 5 – Tribonacci.....	17
Solution Idea.....	17
Code.....	17
Test	18

Table of Figures

Figure 1: fibonacci iterative.....	3
Figure 2: Fibonacci Recursive.....	4
Figure 3: memoized	5
Figure 4: Results of Fibonacci.....	6
Figure 5: Tower of Hanoi.....	7
Figure 6: Tower of Hanoi test.....	9
Figure 7: decimalToBinary.....	10
Figure 8: Binary Conversion test.....	11
Figure 9: Convert number to words.....	15
Figure 10: number conversion test.....	16
Figure 11: Tribonacci	18
Figure 12: tribonacci test	18

Exercise 1 – Fibonacci

Solution Idea

The iterative approach to Fibonacci is simple, just set up the starting variables, then loop and add the previous results.

The recursive approach is also really simple. The base case is $n \leq 1$ and I return the recursive method with $n - 1$ added with $n - 2$.

The memoized approach uses an external array that gets passed along. First I check if n is outside the bounds of the array, then I check if the value for n is already calculated, otherwise I calculate the value and add it to the array.

Code

```
public int fibonacciIterative(int n)
{
    int a = 0, b = 1, c = 1;

    if (n == 0)
        return a;

    if (n == 1)
        return b;

    for (int i = 2; i <= n; i++)
    {
        c = a + b;
        a = b;
        b = c;
    }

    return c;
}
```

Figure 1: fibonacci iterative

```
public int fibonacciRecursive(int n)
{
```

Recursion and Iteration

```
    if (n <= 1)
    {
        return n;
    }

    return fibonacciRecursive(n - 1) + fibonacciRecursive(n -
2);
}
```

Figure 2: Fibonacci Recursive

```
public int fibonacciMemoized(int n, int memoSize)
{
    int[] memo = new int[memoSize];

    return fibonacciMemoized(n, memo);
}

private int fibonacciMemoized(int n, int[] memo)
{
    if (n <= 1)
    {
        return n;
    }

    if (n >= memo.length)
    {
        return fibonacciMemoized(n - 1, memo) +
fibonacciMemoized(n - 2, memo);
    }
    if (memo[n] != 0)
    {
        return memo[n];
    }
    else
    {
        memo[n] = fibonacciMemoized(n - 1, memo) +
fibonacciMemoized(n - 2, memo);
        return memo[n];
    }
}
```

Recursion and Iteration

```
public int fibonacciMemoized(int n)
{
    return fibonacciMemoized(n, n + 1);
}
```

Figure 3: memoized

Test

=== Fibonacci ===

Iterative Approach:

F(8) = 21
F(7) = 13
F(6) = 8
F(5) = 5
F(4) = 3
F(3) = 2
F(2) = 1
F(1) = 1
F(0) = 0

Recursive Approach:

F(8) = 21
F(7) = 13
F(6) = 8
F(5) = 5
F(4) = 3
F(3) = 2
F(2) = 1
F(1) = 1
F(0) = 0

Memoized Approach:

F(8) = 21
F(7) = 13
F(6) = 8
F(5) = 5
F(4) = 3
F(3) = 2

Recursion and Iteration

$F(2) = 1$

$F(1) = 1$

$F(0) = 0$

Time Comparison:

Fibonacci method times for $n = 40$:

Iterative result: 102334155, Time: 1500 ns

Memoized (medium memo : 15) result: 102334155, Time: 1867100 ns

Memoized result ($n + 1$ memo): 102334155, Time: 16800 ns

Memoized (large memo : 30) result: 102334155, Time: 134300 ns

Memoized (small memo : 5) result: 102334155, Time: 56583400 ns

Recursive result: 102334155, Time: 165721100 ns

Figure 4: Results of Fibonacci

As expected, the iterative result is the most efficient. Memoization helps a lot with the recursive solution.

Exercise 2 – Tower of Hanoi

Solution Idea

The recursive Tower of Hanoi solution uses two recursive steps. The first step is to move the top disks away, by alternating between the auxiliary and destination peg. Then it prints out the moves, in reverse, before continuing with the second recursive step to move the smaller disks back on top of the moved larger disk on the destination peg.

Code

```
public TowerOfHanoi(int disks)
{
    setDisks(disks);

    UI.printlnBlue("Solving Tower of Hanoi with " + getDisks() +
" disks:");
    solve(getDisks(), 'A', 'C', 'B');
}

private void solve(int n, char fromPeg, char toPeg, char auxPeg)
{
    if (n == 0)
    {
        return;
    }

    solve(n - 1, fromPeg, auxPeg, toPeg);

    UI.printlnGreen("Move disk " + n + " from peg " + fromPeg +
" to peg " + toPeg);

    solve(n - 1, auxPeg, toPeg, fromPeg);
}
```

Figure 5: Tower of Hanoi

Test

Solving Tower of Hanoi with 3 disks:

```
Move disk 1 from peg A to peg C
Move disk 2 from peg A to peg B
Move disk 1 from peg C to peg B
Move disk 3 from peg A to peg C
Move disk 1 from peg B to peg A
Move disk 2 from peg B to peg C
Move disk 1 from peg A to peg C
```

Solving Tower of Hanoi with 4 disks:

```
Move disk 1 from peg A to peg B
Move disk 2 from peg A to peg C
Move disk 1 from peg B to peg C
Move disk 3 from peg A to peg B
Move disk 1 from peg C to peg A
Move disk 2 from peg C to peg B
Move disk 1 from peg A to peg B
Move disk 4 from peg A to peg C
Move disk 1 from peg B to peg C
Move disk 2 from peg B to peg A
Move disk 1 from peg C to peg A
Move disk 3 from peg B to peg C
Move disk 1 from peg A to peg B
Move disk 2 from peg A to peg C
Move disk 1 from peg B to peg C
```

Solving Tower of Hanoi with 5 disks:

```
Move disk 1 from peg A to peg C
Move disk 2 from peg A to peg B
Move disk 1 from peg C to peg B
Move disk 3 from peg A to peg C
Move disk 1 from peg B to peg A
Move disk 2 from peg B to peg C
Move disk 1 from peg A to peg C
Move disk 4 from peg A to peg B
Move disk 1 from peg C to peg B
Move disk 2 from peg C to peg A
Move disk 1 from peg B to peg A
Move disk 3 from peg C to peg B
Move disk 1 from peg A to peg C
```

Recursion and Iteration

Move disk 2 from peg A to peg B
Move disk 1 from peg C to peg B
Move disk 5 from peg A to peg C
Move disk 1 from peg B to peg A
Move disk 2 from peg B to peg C
Move disk 1 from peg A to peg C
Move disk 3 from peg B to peg A
Move disk 1 from peg C to peg B
Move disk 2 from peg C to peg A
Move disk 1 from peg B to peg A
Move disk 4 from peg B to peg C
Move disk 1 from peg A to peg C
Move disk 2 from peg A to peg B
Move disk 1 from peg C to peg B
Move disk 3 from peg A to peg C
Move disk 1 from peg B to peg A
Move disk 2 from peg B to peg C
Move disk 1 from peg A to peg C

Figure 6: Tower of Hanoi test

Exercise 3 – Decimal to Binary

Solution Idea

Using a `StringBuilder`, I append the modulus of 2 of the decimal number, before recursing using the half of the decimal number. The base case is `decimal == 0` and to check for `decimal == 0`, I first check if the string is empty to return `"0"`.

Code

```
private static String decimalToBinary(int decimal, StringBuilder
binary)
{
    // Base case
    if (decimal == 0)
    {
        // Handle the case where the input decimal number is 0
        if (binary.isEmpty())
        {
            return "0";
        }

        // Reverse the binary string since its constructed
backwards
        return binary.reverse().toString();
    }

    binary.append(decimal % 2);

    return decimalToBinary(decimal / 2, binary);
}
```

Figure 7: decimalToBinary

Test

```
===== Decimal to Binary Converter =====
Decimal: 0 -> Binary: 0
Decimal: 1 -> Binary: 1
Decimal: 5 -> Binary: 101
```

Recursion and Iteration

Decimal: 10 -> Binary: 1010
Decimal: 15 -> Binary: 1111
Decimal: 42 -> Binary: 101010
Decimal: 255 -> Binary: 11111111

Figure 8: Binary Conversion test

Exercise 4 – Number to Words

Solution Idea

I created methods for converting units to words, tens to words and teens to words. In the recursive method, I check the length of the number and appropriately convert the unit, either add thousand or hundred to the string, and handle the number from 99 – 20 and 19 to 10 separately. The base case is if the number is either a teen or a lower than 10, or equal to zero. Each recursive step, I remove the first digit of the number.

Code

```
private static String numberToWords(int number, StringBuilder
str)
{
    if (number < 0 || number > MAX_NUMBER)
    {
        throw new IllegalArgumentException("Number must be
between 0 and " + MAX_NUMBER);
    }

    // thousands

    if (number >= 1000)
    {

str.append(convertNumberToWord(getFirstDigit(number))).append("
Thousand ");

        return numberToWords(removeFirstDigit(number), str);
    }

    // hundreds
    if (number >= 100)
    {

str.append(convertNumberToWord(getFirstDigit(number))).append("
Hundred ");

        return numberToWords(removeFirstDigit(number), str);
    }
}
```

Recursion and Iteration

```
}

// tens
if (number >= 20)
{
    str.append(convertTensToWord(number)).append(" ");
    return numberToWords(removeFirstDigit(number), str);
}

// teens
if (number >= 10)
{
    str.append(convertTeensToWord(number)).append(" ");
    return str.toString().trim();
}

// units
if (number > 0)
{
    str.append(convertNumberToWord(number)).append(" ");
    return str.toString().trim();
}

if (str.isEmpty())
{
    str.append(convertNumberToWord(0)).append(" ");
    return str.toString().trim();
}

return str.toString().trim();
}

private static String convertNumberToWord(int number)
{
    if (number < 0 || number > 9)
    {
        throw new IllegalArgumentException("Number must be
between 0 and 9");
    }

    String[] words = {
        "Zero", "One", "Two", "Three", "Four",
        "Five", "Six", "Seven", "Eight", "Nine"
    }
```

Recursion and Iteration

```
};

return words[number];
}

private static String convertTeensToWord(int number)
{
    if (number < 10 || number > 19)
    {
        throw new IllegalArgumentException("Number must be
between 10 and 19");
    }

    String[] words = {
        "Ten", "Eleven", "Twelve", "Thirteen", "Fourteen",
        "Fifteen", "Sixteen", "Seventeen", "Eighteen",
        "Nineteen"
    };

    return words[number - 10];
}

private static String convertTensToWord(int number)
{
    if (number < 20 || number > 99)
    {
        throw new IllegalArgumentException("Number must be
between 10 and 99");
    }

    String[] words = {
        "Twenty", "Thirty", "Forty", "Fifty", "Sixty",
        "Seventy", "Eighty", "Ninety"
    };

    return words[(number / 10) - 2];
}

private static int getFirstDigit(int number)
{
    while (number >= 10)
    {
```

Recursion and Iteration

```
        number /= 10;
    }
    return number;
}

private static int getLastDigit(int number)
{
    return number % 10;
}

public static int removeFirstDigit(int number)
{
    int divisor = 1;

    while (number / divisor >= 10)
    {
        divisor *= 10;
    }

    return number % divisor;
}
```

Figure 9: Convert number to words

Test

```
=====
=== Number to Words Conversion ===
=====

0      -> Zero
5      -> Five
19     -> Nineteen
23     -> Twenty Three
42     -> Forty Two
100    -> One Hundred
123    -> One Hundred Twenty Three
456    -> Four Hundred Fifty Six
1000   -> One Thousand
1234   -> One Thousand Two Hundred Thirty Four
5678   -> Five Thousand Six Hundred Seventy Eight
```


Recursion and Iteration

9999 -> Nine Thousand Nine Hundred Ninety Nine

Figure 10: number conversion test

Exercise 5 – Tribonacci

Solution Idea

Tribonacci is very similar to Fibonacci, so I just updated the memoized Fibonacci methods for Tribonacci.

Added new base conditions: $n \leq 1 \rightarrow 0$; $n \leq 3 \rightarrow 1$

Code

```
private static int tribonacciMemoized(int n, int[] memo)
{
    if (n <= 1)
    {
        return 0;
    }

    if (n <= 3)
    {
        return 1;
    }

    if (n >= memo.length)
    {
        return tribonacciMemoized(n - 1, memo) +
tribonacciMemoized(n - 2, memo) + tribonacciMemoized(n - 3,
memo);
    }
    if (memo[n] != 0)
    {
        return memo[n];
    }
    else
    {
        memo[n] = tribonacciMemoized(n - 1, memo) +
tribonacciMemoized(n - 2, memo) + tribonacciMemoized(n - 3,
memo);
        return memo[n];
    }
}
```

Figure 11: Tribonacci

Test

```
=====
=== Tribonacci Sequence ===
=====

T(0) = 0
T(1) = 0
T(2) = 1
T(3) = 1
T(4) = 2
T(5) = 4
T(6) = 7
T(7) = 13
T(8) = 24
T(9) = 44
T(10) = 81
T(11) = 149
T(12) = 274
T(13) = 504
T(14) = 927
```

Figure 12: tribonacci test